Ensemble Methods

BSc. Nhat-Tan Bui

Software Engineering Lab (SELab),
University of Science - VNUHCM
Artificial Intelligence and Computer Vision Lab (AICV),
University of Arkansas, Fayetteville
https://tanbuinhat.github.io/

Content

- Ensemble
- Bagging
- Boosting
- AdaBoost
- Gradient Boosting

Ensemble

- Weak learners (e.g., Decision Trees, Naïve Bayes) suffer from the **bias-variance tradeoff**. It means that the model with the low bias has a high variance and vice versa.
  - High bias: **Underfitting** and cannot learn well in the training data.
  - High variance: **Overfitting** and cannot generalize in the unseen data.

- Ensemble: If one weak model cannot solve the problem then take multiple weak models.
  - Bagging (bootstrap aggregation) (Breiman, 1996): reduce variance by **aggregated prediction** over a collection of base learners. Random Forests algorithm is a version of bagging ensemble.
  - Boosting (Freund & Shapire, 1996): reduce both bias and variance by **iteratively reweighted** in training data. Predict by the final classifier.
  - Stacking: reduce both bias and variance by **supervisor model**. The supervisor model will learn to combine the prediction of a base learner collection.

Bagging

- Generate bootstrap samples which have the **same size** of training data.
  - **Randomly selecting** objects in the original data.
  - Bootstrap sample can have **duplicate** objects.

- Each weak model is **trained independently** in each boostrap sample. Thus one model does not affect the learning process of another model.
- The final prediction is decided by **majority vote.**
- Random Forest algorithm extends bagging method by **“de-correlating”**. Specifically, only a **subset of samples** are selected for splitting, unlike the original bagging when **all samples** are considered for splitting at each tree split.

Boosting

- Combine many base models, each one concentrates on **learning the misclassified samples of the previous model**.

The boosting process can be formulated

- Input: $S_N = \{(x_1, y_1), ..., (x_N, y_N)\}$ where N is the number of samples.
- Initialize: all samples have equal weight α
- For each n = 1, ..., N:
  - Train base model by the weighted sample to attain the hypothesis h_n
  - Calculate the weighted error err_n
  - Calculate the weighted hypothesis α_n and update sample weights for next iteration
- Output: $h_n = \sum_{n=1}^{T} \alpha_n h_n(x)$

AdaBoost

AdaBoost optimizes the Exponential Loss. This function is **differentiable and grows exponentially** for negative values, thus the algorithm can **identify the outlier better**.

$$ExpLoss = \frac{1}{N}\sum_{n=1}^{N} exp(-y_n, \widehat{\mathrm{y_n}})$$

where $\widehat{y_n} = h_{n-1}(x_n) + \alpha_n h_n(x_n)$

AdaBoost

The AdaBoost algorithm can be formulated

- Initialize weight $\alpha_n = 1/N$
- At i^{th} step, train model T^i by the weighted sample
- Update the weight

$$\alpha_n = \frac{\alpha_n exp\left(-\lambda_i y_n T^i(x_n)\right)}{Z}$$

where $\lambda_i = \frac{1}{2}\ln\left(\frac{1-\epsilon_i}{\epsilon_i}\right)$ and Z is the normalizing constant

AdaBoost

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score

clf = AdaBoostClassifier () # define estimator and optional parameter
clf.fit(X_train, y_train) # fit model in training data

y_pred = clf.predict(X_test) # predict model in testing data
accuracy = accuracy_score(y_test, y_pred) # evaluate the accuracy of model

print("Accuracy: ", accuracy) # print the accuracy
print("Prediction label: ", y_pred) # print prediction label for test set
```

Gradient Boosting

The Gradient Boosting algorithm, which is the combination of Gradient Descent and Boosting, is a generalization of AdaBoost. This technique allows the model optimizes any arbitrary loss functions (e.g., Square Loss, Logistic Loss, Margin Loss, etc.)

It has many implementation libraries:

- XGBoost: https://xgboost.readthedocs.io/en/stable/
- LGBM: https://lightgbm.readthedocs.io/en/v3.3.2/
- CatBoost: https://catboost.ai/